

When AI Teammates Meet Code Review: Collaboration Signals Shaping the Integration of Agent-Authored Pull Requests

Costain Nachuma
Computer Science Department
Idaho State University
Pocatello, Idaho, USA
costainnachuma@isu.edu

Minhaz Zibran
Computer Science Department
Idaho State University
Pocatello, Idaho, USA
zibran@isu.edu

Abstract

Autonomous coding agents increasingly contribute to software development by submitting pull requests on GitHub; yet, little is known about how these contributions integrate into human-driven review workflows. We present a large empirical study of agent-authored pull requests using the public AIDev dataset, examining integration outcomes, resolution speed, and review-time collaboration signals. Using logistic regression with repository-clustered standard errors, we find that reviewer engagement has the strongest correlation with successful integration, whereas larger change sizes and coordination-disrupting actions, such as force pushes, are associated with a lower likelihood of merging. In contrast, iteration intensity alone provides limited explanatory power once collaboration signals are considered. A qualitative analysis further shows that successful integration occurs when agents engage in actionable review loops that converge toward reviewer expectations. Overall, our results highlight that the effective integration of agent-authored pull requests depends not only on code quality but also on alignment with established review and coordination practices.

CCS Concepts

• **Software and its engineering** → **Empirical software engineering**; *Collaboration in software development*; *Software maintenance tools*; • **Computing methodologies** → *Artificial intelligence*.

Keywords

AI, pull requests, autonomous coding agents, merge, acceptance, collaboration, code review, code churn, GitHub, empirical study

ACM Reference Format:

Costain Nachuma and Minhaz Zibran. 2026. When AI Teammates Meet Code Review: Collaboration Signals Shaping the Integration of Agent-Authored Pull Requests. In *23rd International Conference on Mining Software Repositories (MSR '26)*, April 13–14, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 5 pages. <https://doi.org/10.1145/3793302.3793561>

1 Introduction

Autonomous coding agents are increasingly participating in everyday software development [13]. Beyond acting as code generation tools, these systems now submit pull requests that fix bugs, add features, and extend test suites on GitHub. As a result, human

developers are routinely asked to review, carefully interpret, and integrate changes authored by non-human contributors [2, 7].

Recent large-scale studies based on the AIDev dataset document the rapid growth of agent-authored pull requests and their aggregate merge rates across projects [6, 7]. However, we note that adoption alone does not imply effective collaboration. In pull-based development, integration outcomes reflect not only the technical correctness of a change but also how contributors engage with review workflows, iteration cycles, and coordination norms [3, 11, 12]. Whether autonomous agents can function as effective collaborators within these socio-technical processes, rather than merely producing correct patches, remains unclear.

Two gaps motivate our study. First, while agent-authored pull requests are becoming increasingly common, we lack a clear understanding of how reliably they integrate and how long it takes to reach a decision. Second, beyond agent identity, there is limited empirical evidence on which review-time collaboration signals, such as iteration behavior, reviewer engagement, testing activity, and coordination stability, are associated with successful integration. Despite extensive prior work on pull-based development [3, 11, 12], there is limited empirical evidence on how autonomous agents engage with these review-time collaboration processes.

Addressing these gaps is essential as software development shifts toward mixed human-AI teams. For agents to serve as effective collaborators rather than isolated patch generators, they must align with the implicit social and procedural expectations of code review. In this paper, we present a large-scale empirical analysis of agent-authored pull requests, focusing on how review-time collaboration dynamics shape integration outcomes. Rather than treating these pull requests solely as code artifacts, we analyze their integration as a socio-technical process mediated by human review and coordination. We examine the following two research questions:

RQ1: *Among agent-authored pull requests, what proportions are merged, closed without merging, or remain open, and how long do resolved pull requests take to reach a decision?*

Measures: share(merged), share(closed-unmerged), share(open), and time-to-decision (mean and median).

RQ2: *Which review-time collaboration signals are associated with the integration of agent-authored pull requests?*

Focus: iteration intensity, testing behavior, reviewer engagement, and coordination stability (from GitHub review-time traces).

Signals: number of commits (bucketed), Δ LOC and number of files changed, a tests-added indicator, a force-push indicator, review presence, and time to first review.



This work is licensed under a Creative Commons Attribution 4.0 International License. *MSR '26, Rio de Janeiro, Brazil*

© 2026 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-2474-9/26/04

<https://doi.org/10.1145/3793302.3793561>

2 Dataset and Operational Definitions

Dataset snapshot. We use the **AIDev** dataset of agent-authored pull requests (PRs) from Zenodo (version 3; accessed Nov 2025), citing both the dataset record and its accompanying preprint [6, 7].

Acquisition and binding. All Parquet tables were obtained from Zenodo version 3 and bound as DuckDB views using an in-memory DuckDB engine. The tables used include *pull_request*, *pr_task_type*, *pr_commits*, *pr_commit_details*, *pr_reviews*, *pr_timeline*, *repository*, and *user*.

Dataset scope (curated popular). Our analysis focuses on the curated subset of popular repositories with at least 100 GitHub stars, following the dataset authors' recommended filtering. This subset contains 33,596 agent-authored PRs submitted by 1,797 distinct developers across 2,807 repositories, spanning multiple coding agents.

Agent-authored PRs. A PR is considered agent-authored if it appears in *pr_task_type* with a non-null agent label. Unless otherwise noted, all reported statistics and analyses are computed over this set. The dataset does not distinguish between autonomous and human-invoked agent submissions; we therefore treat all agent-authored pull requests uniformly.

Outcomes and timestamps. PR outcomes are derived directly from GitHub timestamps to avoid ambiguity:

$$\text{merged} := (\text{merged_at} \neq \emptyset),$$

$$\text{closed_without_merge} := (\text{closed_at} \neq \emptyset \wedge \text{merged_at} = \emptyset),$$

$$\text{open} := (\text{closed_at} = \emptyset \wedge \text{merged_at} = \emptyset).$$

For resolved PRs, the decision time is taken as the merge time when present and otherwise the closure time. All timestamps are normalized to UTC.

Time to decision. We compute time to decision as the elapsed time between *created_at* and the decision timestamp for resolved PRs. Due to right-skewed lifetimes, we report both mean and median values.

Measures. We report the share of agent-authored PRs that are merged, closed without merge, or remain open. We use Wilson score confidence intervals (95%), which provide more reliable coverage than normal approximations for binomial proportions, particularly near boundary values [1, 10].

Replication package. We release a complete replication package containing all scripts required to reproduce our results [9]. RQ1 is reproduced by running `python run_rq1.py --config config.yaml`. RQ2 is reproduced by first constructing features and then fitting regression models using the provided scripts.

3 Integration and Resolution (RQ1)

3.1 Motivation

Pull requests (PRs) are the dominant mechanism for integrating changes in modern, pull-based development, and prior work has shown that PR outcomes (merge vs. non-merge) reflect both technical and social processes during review [3, 11, 12]. As autonomous coding agents increasingly submit PRs at scale [6, 7], it is important to establish a baseline: (i) whether agent-authored PRs integrate at rates comparable across agents, and (ii) how quickly they reach a decision. This baseline anchors subsequent analyses.

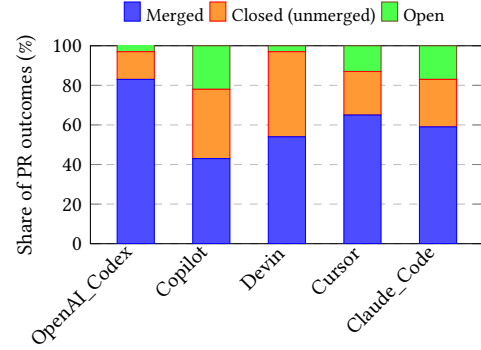


Figure 1: Outcomes of agent-authored pull requests

3.2 Approach

We study agent-authored PRs from the AIDev dataset [6, 7]. For each PR, we derive its outcome directly from GitHub timestamps to avoid ambiguity in API state fields: a PR is *merged* if *merged_at* is present; *closed-unmerged* if *closed_at* is present and *merged_at* is absent; otherwise it is *open*. We report outcome shares overall and per agent, along with 95% confidence intervals for proportions using standard interval estimation methods [1, 10].

To characterize resolution dynamics, we compute *time-to-decision* for resolved PRs (merged or closed-unmerged) as the elapsed time from *created_at* to *merged_at* (if merged) or *closed_at* (if closed-unmerged). We summarize decision latency using both the mean and median to account for skew common in PR lifetimes.

Exploratory note on reverts. Reverts can be an imperfect proxy for post-merge problems, as repository-level signals may arise from refactoring, redesign, or other maintenance activities rather than defects [4]. Accordingly, we do not consider reverts as a primary outcome and focus our analysis on review-time integration and collaboration dynamics.

3.3 Results

Across 33,596 agent-authored PRs spanning 5 coding agents, 71.5% were merged, 21.6% were closed without merge, and 6.9% remained open at the time of data collection (merged share 95% CI: [0.710, 0.720]). These aggregates indicate that most agent-authored PRs reach integration, but a substantial fraction still fail to merge or remain unresolved.

3.3.1 Outcome rates vary sharply by agent. Merge rates differ markedly across agents, suggesting heterogeneity in both PR quality and fit to reviewer expectations. For example, OpenAI_Codex shows the highest merge share (82.6%), while Copilot merges substantially less often (43.0%); Devin lies in between (53.8%). Importantly, the *open* share also differs across agents (e.g., Copilot exhibits a notably higher open fraction than others), indicating different resolution dynamics and/or triage practices.

3.3.2 Decision latency differs even more than merge rates. Resolution speed varies substantially by agent. For resolved PRs, OpenAI_Codex reaches a decision extremely quickly (median <1 hour; mean ≈ 19.4 hours), while Copilot and Devin are slower (median 13 hours and 9 hours, respectively; with mean times exceeding

80–100 hours). This pattern suggests differences in review effort and alignment with repository norms.

3.4 Summary

RQ1 establishes a baseline for agentic PR integration and resolution dynamics. Overall, a majority of agent-authored PRs merge, but integration success and decision speed vary strongly across agents. These differences motivate our subsequent analyses of review-time collaboration signals: beyond whether an agent can produce working code, the interaction and coordination costs borne by human reviewers may be a key determinant of whether and how quickly agent-authored PRs integrate [3, 11, 12].

Answer to RQ1: Most agent-authored pull requests are successfully merged, but both integration rates and resolution speed vary substantially across agents. These differences indicate that agentic PR integration depends not only on code correctness, but also on how agents interact with human review workflows and manage coordination costs during review.

4 Collaboration Signals (RQ2)

4.1 Motivation

RQ1 revealed substantial variation in integration outcomes and decision latency across coding agents. Beyond agent identity, this raises the question of which *review-time collaboration signals* are associated with the successful integration of agent-authored pull requests (PRs). Prior work shows that reviewers rely on both technical and social cues, such as change size, reviewer participation, and coordination stability when evaluating pull requests [3, 11, 12]. We examine whether similar signals shape the acceptance of agent-authored PRs.

4.2 Approach

We model pull request integration as a binary outcome (merged vs. closed without merge). We focus on *review-time collaboration signals* observable in GitHub traces prior to the final decision, capturing how agent-authored pull requests align with review workflows through iteration and change magnitude, coordination stability, and reviewer engagement. We estimate interpretable associations between these signals and merge decisions using logistic regression. Predictors capture three dimensions: (1) iteration and change magnitude, (2) coordination stability and process signals, and (3) reviewer engagement. Agent indicators are included to control for systematic differences across coding agents.

Change magnitude is measured using log-transformed code churn ($\log(1 + \Delta\text{LOC})$) and the number of files modified. Coordination stability is captured via a binary indicator for force pushes during review. Testing behavior is represented by a binary indicator flagging whether test files were modified. Reviewer engagement is measured using indicators for whether a pull request received at least one review and the log-transformed time to first review. Iteration intensity is operationalized either through commit-count buckets (Model A) or log-transformed commit counts (Model B), following prior work on code review and pull request integration [8].

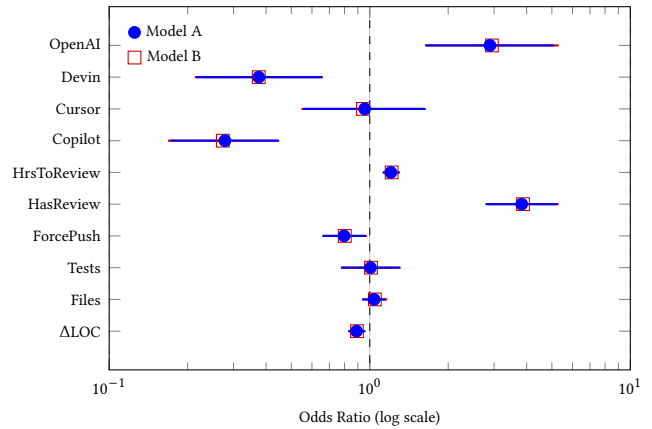


Figure 2: Forest plot of logistic regression predictors

To address skew, size-related variables are log-transformed using $\log(1 + x)$. We estimate logistic regression models with repository-clustered standard errors and report odds ratios. Results are interpreted as associations rather than causal effects. Claude Code is used as the reference category for agent indicators.

4.3 Results

Figure 2 reports odds ratios from logistic regression models. Model A uses commit-count buckets, while Model B uses log-transformed commit counts to operationalize iteration intensity. Consistent effects across models indicate robustness. The x-axis shows odds ratios on a logarithmic scale.

4.3.1 Reviewer engagement dominates integration outcomes. Across both models, receiving at least one review is the strongest correlate of merging. Agent-authored PRs that receive reviewer attention exhibit substantially higher odds of integration, suggesting that reviewers selectively invest effort in changes perceived as viable.

4.3.2 Coordination-disrupting behaviors are penalized. Force pushes during review are consistently associated with lower merge likelihood; while sometimes required for repository hygiene (e.g., squash or rebase policies), history rewriting during active review can disrupt shared understanding and increase coordination costs for reviewers.

4.3.3 Change size increases review burden. Larger changes are less likely to be merged even after controlling for agent identity and reviewer engagement. This aligns with prior PR research and suggests that reviewers apply similar risk heuristics to agent- and human-authored contributions.

4.3.4 Iteration and testing show limited independent effects. Neither iteration intensity nor test additions are significantly associated with merge outcomes once reviewer engagement and coordination stability are accounted for. Producing more revisions or adding tests alone does not increase integration likelihood without reducing reviewer uncertainty.

4.3.5 Time-to-review reflects workflow dynamics. Conditional on receiving a review, longer time to first review is positively associated with merging, likely reflecting repository-level prioritization rather than a direct benefit of delayed review.

4.3.6 Synthesis. Overall, reviewers reward agent behaviors that reduce coordination cost and penalize those that disrupt shared understanding. Successful integration depends less on activity volume and more on alignment with the social and procedural norms of code review. Comparing these patterns with fully human-authored pull requests is an important direction for future work.

4.4 Qualitative Follow-up

While the regression analysis identifies which review-time collaboration signals are associated with integration outcomes, it does not explain how these signals manifest during review. We therefore analyzed a random and purposive sample of 60 agent-authored pull requests spanning merged and non-merged outcomes. Using a pre-defined codebook derived from prior pull request review literature and pilot inspection of agent-authored review threads, each pull request was assigned a single primary code capturing the dominant review-time mechanism influencing its outcome. Disagreements were resolved through discussion. Table 1 summarizes the results.

4.4.1 Actionable review convergence enables success. The dominant success pattern is an *actionable review loop*, accounting for 32 PRs and 30 of all merged cases. In these cases, reviewers provided concrete feedback and agents responded with targeted revisions that converged toward acceptance, explaining the strong positive association between reviewer engagement and merging.

4.4.2 Design conflict and coordination breakdown explain rejection. All non-merged PRs are associated with failure mechanisms. Design or architectural disagreements reflect misalignment with project principles, while coordination breakdowns (e.g., force pushes) disrupt review context. These patterns directly correspond to negative associations observed in the regression models.

4.4.3 Incomplete or policy-blocked changes rarely recover. PRs coded as incomplete solutions or blocked by process constraints (e.g., policy or repository state) consistently failed to merge, clarifying why iteration alone does not improve outcomes once engagement and coordination stability are considered.

4.4.4 Iteration without convergence is insufficient. Consistent with quantitative results, additional commits or test additions do not lead to integration unless they directly address reviewer concerns and reduce uncertainty.

4.4.5 Synthesis. Taken together, the qualitative findings show that reviewer engagement is effective only when agents participate in stable, goal-aligned review interactions. Integration success depends on convergence through feedback rather than activity volume.

Answer to RQ2: Integration of agent-authored pull requests is driven primarily by review-time collaboration signals rather than iteration volume. Reviewer engagement increases merge likelihood when it produces actionable feedback and convergence, while large changes and coordination-disrupting behaviors reduce success. Iteration alone has limited impact without alignment with reviewer expectations and project norms.

Across RQ1 and RQ2, our results show that integration of agent-authored pull requests is shaped primarily by review-time coordination rather than iteration volume. Quantitative models identify reviewer engagement and coordination stability as dominant signals, while the qualitative analysis explains how convergence through

Table 1: Qualitative outcomes of agent-authored pull requests

Primary Driver	#	Merged	Not Merged	Primary Outcome
Actionable review loop	32	30	2	Success
Design disagreement	10	0	10	Failure
Incomplete solution	7	0	7	Failure
Process or policy issue	3	0	3	Failure
Coordination break	2	0	2	Failure
Incorrect or failing CI	2	0	2	Failure
Other (scope, style, or stalled discussion)	4	0	4	Failure
Total	60	30	30	

actionable feedback leads to successful merges. Conversely, design misalignment, incomplete solutions, and coordination disruptions consistently explain rejection. These findings highlight the need for alignment with established review practices.

5 Related Work

Prior research characterizes pull request evaluation as a socio-technical process in which integration depends on both technical and social factors. Studies show that characteristics such as change size influence acceptance, while review-time interaction plays a central role in integration decisions [3, 11, 12]. Reviewer engagement, discussion dynamics, and coordination practices help reviewers assess trust, coordination cost, and perceived risk, often beyond code correctness alone.

Recent advances in large language models have enabled autonomous coding agents to submit pull requests at scale. The AIDev dataset provides the first systematic evidence of this shift, documenting adoption patterns and aggregate merge outcomes [6, 7]. However, existing studies largely treat agent-authored pull requests as technical artifacts, leaving open how they participate in established socio-technical review processes. Building on this literature, our study examines whether review-time collaboration signals that shape human pull request integration similarly influence agent-authored contributions.

6 Threats to Validity

Construct validity. We infer collaboration from GitHub artifacts (e.g., reviews, commits, force pushes), which may not fully capture reviewer intent or interaction quality [5]. We mitigate this through complementary qualitative coding using a predefined codebook and discussion-based resolution of disagreements.

Internal validity. The study is observational and does not support causal claims. Although we control for agent identity and repository effects, unobserved project norms or reviewer practices may influence outcomes.

External validity. Our analysis focuses on agent-authored pull requests in large public GitHub repositories; findings may not generalize to private projects, smaller communities, or future agents with different behaviors.

7 Conclusion

This paper studied how agent-authored pull requests integrate into human-driven development workflows using the AIDev dataset. We show that integration success depends primarily on review-time collaboration signals rather than iteration volume. Reviewer engagement increases merge likelihood when it leads to actionable

feedback and convergence, while larger changes and coordination-disrupting behaviors reduce success. These findings highlight that effective agentic software engineering requires alignment with established code review and coordination practices.

To facilitate verification and replication of this work, a replication package [9] is made publicly available.

References

- [1] Alan Agresti and Brent A. Coull. 1998. Approximate Is Better Than “Exact” for Interval Estimation of Binomial Proportions. *The American Statistician* 52, 2 (1998), 119–126.
- [2] Cognition Labs. 2024. Devin: The First AI Software Engineer. *arXiv preprint arXiv:2403.08994* (2024).
- [3] Georgios Gousios, Martin Pinzger, and Arie van Deursen. 2014. An Exploratory Study of the Pull-Based Software Development Model. In *Proceedings of the 36th International Conference on Software Engineering (ICSE)*. 345–355. doi:10.1145/2568225.2568260
- [4] Kim Herzig, Sascha Just, and Andreas Zeller. 2013. It’s Not a Bug, It’s a Feature: How Misclassification Impacts Bug Prediction. In *Proceedings of the 10th Working Conference on Mining Software Repositories (MSR)*. 392–401.
- [5] Eirini Kalliamvakou, Georgios Gousios, Kelly Blincoe, Leif Singer, Daniel M. German, and Daniela Damian. 2016. An In-Depth Study of the Promises and Perils of Mining GitHub. *Empirical Software Engineering* 21, 5 (2016), 2035–2071. doi:10.1007/s10664-015-9393-5
- [6] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. *AIDev: Studying Coding Agents on GitHub*. doi:10.5281/zenodo.16919272
- [7] Hao Li, Haoxiang Zhang, and Ahmed E. Hassan. 2025. The Rise of AI Teammates in Software Engineering (SE) 3.0: How Autonomous Coding Agents Are Reshaping Software Engineering. *arXiv:2507.15003 [cs.SE]* <https://arxiv.org/abs/2507.15003>
- [8] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2014. The Impact of Code Review Coverage and Code Review Participation on Software Quality. In *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*. 192–201. doi:10.1145/2597073.2597076
- [9] Costain Nachuma and Minhaz Zibran. 2026. *Replication Package for MSR 2026: Evaluating AI Teammates*. doi:10.5281/zenodo.17860503
- [10] Robert G. Newcombe. 1998. Two-Sided Confidence Intervals for the Single Proportion: Comparison of Seven Methods. *Statistics in Medicine* 17, 8 (1998), 857–872.
- [11] Peter C. Rigby and Christian Bird. 2013. Peer Review on Open-Source Software Projects. In *Proceedings of the 2013 International Conference on Software Engineering*. 541–550.
- [12] Jason Tsay, Laura Dabbish, and James Herbsleb. 2014. Let’s Talk About It: Evaluating Contributions through Discussion in GitHub. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on the Foundations of Software Engineering (FSE)*. 144–154. doi:10.1145/2635868.2635880
- [13] Mairieli Wessel, Bruno Mendes de Souza, Igor Steinmacher, Igor S. Wiese, Ivanilton Polato, Ana Paula Chaves, and Marco A. Gerosa. 2018. The Power of Bots: Characterizing and Understanding Bots in OSS Projects. *Proc. ACM Hum.-Comput. Interact.* 2, CSCW, Article 182 (2018). doi:10.1145/3274451